

26-1 수쿱 Official Solution

출제자 - 김수성

문제		난이도
A	문자열 관찰	Easy
B	자릿수의 합	Easy
C	수열 매칭	Normal
D	비순환 그래프	Normal
E	숫자 연쇄 소거	Normal
F	로봇 이동시키기	Hard
G	3 사이클	Hard

A. 문자열 관찰

Implementation

난이도 - Easy

제출 13회, 정답 5명 (정답률 38.4%)

첫 정답 woojin, 3분 51초

A. 문자열 관찰

풀이

문자열의 각 문자를 하나씩 확인합니다.

문자가 'A' 이상 'Z' 이하라면 대문자,
'a' 이상 'z' 이하라면 소문자,
그 외의 문자는 숫자로 세면 됩니다.

문자열은 대문자, 소문자, 숫자로만 이루어져 있으므로
별도의 예외 처리는 필요하지 않습니다.

시간 복잡도는 $O(|S|)$ 입니다.

B. 자릿수의 합

Bruteforce

난이도 - Easy

제출 19회, 정답 4명 (정답률 21%)

첫 정답 woojin, 10분 41초

B. 자릿수의 합

풀이

각 정수 i 에 대해 자릿수의 합을 구한 뒤,
그 값이 7로 나누어떨어지는지 확인합니다.

N 의 최댓값은 100,000이고,
모든 테스트 케이스에서 N 의 합도 200,000 이하이므로
1부터 N 까지 직접 확인해도 충분합니다.

자릿수의 합은 10으로 나눈 나머지를 계속 더하면서 구할 수 있습니다.

시간 복잡도는 $O(\text{자릿수} * N)$ 입니다.

C. 수열 매칭

Sorting, Greedy

난이도 - Normal

제출 13회, 정답 4명 (정답률 30.7%)
첫 정답 songjihoo, 17분 40초

C. 수열 매칭

풀이

수열을 오름차순으로 정렬합니다.

정렬된 수열을 $a_1 \leq a_2 \leq \dots \leq a_n$ 이라고 합니다.

최솟값은 인접한 두 원소끼리 묶으면 됩니다.

$(a_1, a_2), (a_3, a_4), \dots$

최댓값은 작은 절반과 큰 절반을 묶으면 됩니다.

$(a_1, a_{\{N/2+1\}}), (a_2, a_{\{N/2+2\}}), \dots$

정렬에 $O(N \log N)$, 계산에 $O(N)$ 이 걸립니다.

C. 수열 매칭

최솟값 증명

$a_1 \leq a_2 \leq \dots \leq a_n$ 이라고 합시다.

정렬된 네 원소 $a \leq b \leq c \leq d$ 를 생각합니다.

a 와 b 가 서로 묶이지 않은 경우는 다음 둘입니다.

$(a, c), (b, d) / (a, d), (b, c)$

두 경우에 대해 $(a, b), (c, d)$ 와의 비용의 차이는 다음과 같습니다.

$$[(c-a)+(d-b)] - [(b-a)+(d-c)] = 2(c-b) \geq 0$$

$$[(d-a)+(c-b)] - [(b-a)+(d-c)] = 2(c-b) \geq 0$$

따라서 두 쌍을 $(a,b), (c,d)$ 로 바뀌어도 비용은 증가하지 않습니다.

C. 수열 매칭

최솟값 증명

임의의 최적해에서 가장 작은 원소 a_1 을 봅시다.

a_1 이 a_2 와 묶이지 않았다면,
 a_1 과 a_2 가 속한 두 쌍에 앞의 교환을 적용합니다.

그러면 비용을 증가시키지 않고
 a_1 과 a_2 를 한 쌍으로 만들 수 있습니다.

따라서 (a_1, a_2) 를 포함하는 최적해가 존재합니다.

같은 논리를 남은 원소에 반복하면 $(a_1, a_2), (a_3, a_4), \dots$ 가 최적입니다.

C. 수열 매칭

최댓값 증명

$k = N / 2$ 라고 합시다.

임의의 매칭 M 의 각 쌍을 $(x_i, y_i), x_i \leq y_i$ 로 둡니다.

$$\text{cost}(M) = \sum (y_i - x_i) = \sum y_i - \sum x_i$$

$\{y_i\}$ 는 k 개의 원소이므로 $\sum y_i \leq a_{k+1} + \dots + a_n$,

$\{x_i\}$ 는 k 개의 원소이므로 $\sum x_i \geq a_1 + \dots + a_k$ 입니다.

따라서 $\text{cost}(M) \leq (a_{k+1} + \dots + a_n) - (a_1 + \dots + a_k)$ 입니다.

이 값은 $(a_1, a_{k+1}), \dots, (a_k, a_n)$ 으로 달성되므로 최댓값 매칭입니다.

D. 비순환 그래프

Graph, DFS

난이도 - Normal

제출 10회, 정답 3명 (정답률 30%)

첫 정답 songjihoo, 39분 6초

D. 비순환 그래프

풀이

각 연결 컴포넌트별로 생각해봅시다.

정점이 V 개인 연결 컴포넌트를 사이클이 없도록 만들려면
최종적으로 간선은 최대 $V - 1$ 개만 남길 수 있습니다.

따라서 전체 그래프에서 C 를 연결 컴포넌트의 개수라고 하면
비순환 그래프에 남길 수 있는 간선의 최댓값은
 $N - C$ 입니다.

D. 비순환 그래프

풀이

처음 간선의 개수가 M 개이고,
최대로 남길 수 있는 간선의 개수가 $N - C$ 개이므로
삭제해야 하는 최소 간선 수는 다음과 같습니다.

$$M - (N - C) = M - N + C$$

C 는 DFS, BFS 또는 DSU로 구할 수 있습니다.

시간 복잡도는 $O(N + M)$ 입니다.

E. 숫자 연쇄 소거

Stack, Simulation

난이도 - Normal

제출 14회, 정답 2명 (정답률 14.3%)

첫 정답 songjihoo, 1시간 30분 3초

E. 숫자 연쇄 소거

서브 태스크 1

모든 연산에서 $v = 1$ 입니다.

따라서 매 연산마다 x 를 하나만 수열의 뒤에 추가하면 됩니다.

수열을 스택에 직접 저장하고,
추가 직후 뒤에서부터 연속된 x 의 개수를 셉니다.

개수가 x 개라면 뒤의 x 개 원소를 제거합니다.
연쇄 소거가 가능하므로 이 과정을 반복합니다.

수열 길이는 최대 $O(N)$ 이므로 시간 복잡도는 $O(N^2)$ 입니다.

E. 숫자 연쇄 소거

서브 태스크 2

N은 작지만 v 는 1보다 클 수 있습니다.

총 추가 횟수는 최대 $2,000 \times 10,000 = 20,000,000$ 번입니다.

따라서 x 를 v 번 하나씩 추가하는 시뮬레이션도 가능합니다.
단, 매번 뒤에서부터 다시 세면 시간 초과가 날 수 있습니다.

스택에 마지막 연속 구간을 (값, 개수)로 저장합니다.
 x 를 하나 추가할 때 top의 값이 x 이면 개수만 1 증가시키고,
개수가 x 가 되면 해당 구간을 pop합니다.

시간 복잡도는 $O(\sum v)$ 입니다.

E. 숫자 연쇄 소거

서브 태스크 3

전체 수열을 저장할 필요는 없습니다.
같은 수가 연속된 구간을 하나의 묶음으로 압축해서
스택에 저장합니다.

스택의 원소를 (값, 개수)라고 합시다.

새 연산 (x, v) 가 들어왔을 때,
스택의 top 값이 x 라면 개수에 v 를 더하고,
아니라면 (x, v) 를 새로 push합니다.

E. 숫자 연쇄 소거

서브 태스크 3

top의 값이 x 이고 개수가 c 라고 합시다.

$c \geq x$ 라면 맨 뒤의 x 개가 사라집니다.

이 과정을 반복하면 남는 개수는 $c \% x$ 입니다.

따라서 한 번에 다음과 같이 처리할 수 있습니다.

제거되는 개수 = $\text{floor}(c / x) * x$

남는 개수 = $c \% x$

남는 개수가 0이면 top을 pop합니다.

E. 숫자 연쇄 소거

서브 태스크 3

수열의 현재 길이 `result`를 따로 관리합니다.

추가할 때 `result += v`,
소거할 때 제거된 개수만큼 `result`를 줄입니다.

각 연산 후 `result`를 출력하면 됩니다.

각 묶음은 스택에 한 번 들어가고 제거될 때 한 번 나가므로
전체 시간 복잡도는 $O(N)$ 입니다.

F. 로봇 이동시키기

DP

난이도 - Hard

제출 1회, 정답 0명 (정답률 0%)

첫 정답 -

F. 로봇 이동시키기

서브 태스크 1

두 로봇의 위치를 모두 상태로 두는 DP를 생각할 수 있습니다.

$DP[x1][y1][x2][y2] =$

첫 번째 로봇이 $(x1, y1)$, 두 번째 로봇이 $(x2, y2)$ 에 있을 때 얻을 수 있는 최대 점수

가능한 이전 상태는 각 로봇이 위 또는 왼쪽에서 온 경우이므로 총 4가지입니다.

시간 복잡도는 $O(N^4)$ 입니다.

F. 로봇 이동시키기

서브 태스크 2

두 로봇은 매 이동마다 동시에 한 칸씩 움직입니다.

따라서 같은 시점에 두 로봇의 $x + y$ 값은 항상 같습니다.

$t_i = x + y$ 라고 두면,
첫 번째 로봇의 위치는 $(x_1, t_i - x_1)$,
두 번째 로봇의 위치는 $(x_2, t_i - x_2)$ 로 표현할 수 있습니다.

그러면 상태를 3차원으로 줄일 수 있습니다.

F. 로봇 이동시키기

서브 태스크 2

$DP[ti][x1][x2] =$

$x + y = ti$ 인 시점에서 두 로봇의 행이 각각 $x1, x2$ 일 때 얻을 수 있는 최대 점수

$y1 = ti - x1, y2 = ti - x2$ 라고 합시다.

이전 상태는 다음 네 가지입니다.

$(x1, x2), (x1 - 1, x2),$

$(x1, x2 - 1), (x1 - 1, x2 - 1)$

네 값 중 최댓값에 $|A[x1][y1] - A[x2][y2]|$ 를 더합니다.

F. 로봇 이동시키기

서브 태스크 2

시작 상태는 $DP[2][1][1] = 0$ 입니다.

두 로봇이 처음에는 아직 이동하지 않았으므로 점수를 얻지 않습니다.

정답은 $DP[2N][N][N]$ 입니다.

y_1, y_2 가 1 이상 N 이하가 아닌 상태는 불가능한 상태로 무시합니다.

t_i 는 $O(N)$, x_1 과 x_2 도 각각 $O(N)$ 이므로 전체 시간 복잡도는 $O(N^3)$ 입니다.

G. 3 사이클

Tree, DFS, Combinatorics

난이도 - Hard

제출 1회, 정답 0명 (정답률 -%)

첫 정답 -

G. 3 사이클

서브 태스크 1

N 이 작기 때문에 추가할 수 있는 모든 정점 쌍 (u, v) 을 확인할 수 있습니다.

트리에서 u 와 v 사이의 경로를 찾고,
간선을 추가한 뒤 만들어지는 사이클의 모든 정점 차수가
3인지 검사합니다.

모든 쌍을 확인하면 정답을 구할 수 있습니다.
시간 복잡도는 $O(N^3)$ 입니다.

G. 3 사이클

서브 태스크 2

간선 (u, v) 를 추가하면 u 와 v 의 차수는 1 증가하고,
 $u-v$ 경로의 내부 정점들의 차수는 변하지 않습니다.

따라서 조건을 만족하려면 다음이 반드시 필요합니다.

추가하는 간선의 양 끝점 u, v 의 기존 차수는 2
경로 내부 정점들의 기존 차수는 3

G. 3 사이클

서브 태스크 2

차수가 3인 정점들만 모아서 연결 컴포넌트를 생각합니다.

어떤 차수 3 컴포넌트에 인접한 차수 2 정점의 개수를 cnt 라고 합시다.

이 cnt 개의 정점 중 서로 다른 두 정점을 골라 간선을 추가하면, 두 정점 사이의 경로 내부는 모두 차수 3 정점이 됩니다.

따라서 이 컴포넌트가 만드는 정답은 $\text{cnt} * (\text{cnt} - 1) / 2$ 입니다.

G. 3 사이클

서브 태스크 2

모든 차수 3 컴포넌트에 대해 cnt 를 구한 뒤,
 $\text{cnt} \times (\text{cnt} - 1) / 2$ 를 정답에 더해주면 됩니다.

유효한 간선은 내부에 포함되는 차수 3 컴포넌트가
하나로 정해지므로 중복 계산되지 않습니다.

각 정점과 간선을 한 번씩만 확인하면 되므로
시간 복잡도는 $O(N)$ 입니다.